Cahier des Charges

Compresseur MAL

MaxAdLe Compression

Projet de Traitement du Signal

Développement d'un compresseur personnalisé

en Rust avec format d'archive `.mal`

Équipe de développement :

Maxime
Adrien
Léos

25 janvier 2026

Table des matières

1 Introduction **2**

2 État de l'Art **2**
2.1 ZIP et l'algorithme DEFLATE 2
2.2 GZIP 3
2.3 7-Zip et LZMA 3
2.4 Zstandard (Zstd) 3
2.5 Brotli 4

3 Choix Technologique Retenu **4**
3.1 Justification du choix 4
3.2 Technologies complémentaires 5

4 Répartition des Tâches et Planning **5**
4.1 Phase 1 : Jusqu'à la première soutenance 5
4.2 Phase 2 : Jusqu'à la deuxième soutenance 6

5 Conclusion **6**

1 Introduction

La compression de données est un enjeu fondamental en informatique moderne, permettant d'optimiser le stockage et la transmission d'informations. Que ce soit pour archiver des documents, partager des fichiers volumineux ou réduire l'empreinte disque de systèmes de production, les algorithmes de compression sans perte jouent un rôle crucial.

Dans le cadre de ce projet, nous proposons de développer un compresseur de fichiers personnalisé en Rust, avec une extension propriétaire `.mal` (MaxAdLe). Notre objectif est double : d'une part, approfondir notre compréhension des algorithmes de compression classiques (LZ77, Huffman, DEFLATE) ; d'autre part, concevoir une solution complète et ergonomique offrant des fonctionnalités avancées telles que la compression multi-fichiers, la vérification d'intégrité et le suivi en temps réel.

Ce cahier des charges présente l'état de l'art des technologies de compression existantes, justifie nos choix techniques, et détaille la planification du projet ainsi que la répartition des tâches au sein de l'équipe.

2 État de l'Art

Nous avons identifié plusieurs technologies de compression sans perte largement utilisées dans l'industrie. Cette section présente leurs principes, capacités et limites.

2.1 ZIP et l'algorithme DEFLATE

ZIP / DEFLATE

Principe : Le format ZIP utilise l'algorithme DEFLATE, combinaison de LZ77 (compression par dictionnaire) et de codage de Huffman dynamique.

Capacités :

- Compression efficace sur la plupart des types de fichiers texte et binaires
- Support multi-fichiers avec structure d'archive
- Largement supporté par tous les systèmes d'exploitation
- Ratio compression/vitesse équilibré
- Niveaux de compression ajustables (0-9)

Limites :

- Moins performant que des algorithmes modernes (LZMA, Zstandard) en termes de ratio
- Pas de compression parallèle native
- Métadonnées limitées (pas de checksums SHA-256 standard)
- Format fermé avec extensions propriétaires

2.2 GZIP

GZIP

Principe : GZIP utilise également DEFLATE mais pour la compression de fichiers uniques, principalement utilisé pour la compression de flux.
Capacités :

— Très rapide en compression et décompression

— Faible consommation mémoire

— Standard de facto pour la compression HTTP

— CRC32 intégré pour vérification d'intégrité

Limites :

— Fichier unique uniquement (pas d'archivage)

— Ratio de compression inférieur à LZMA ou Brotli

— Pas de support multi-threading

2.3 7-Zip et LZMA

7-Zip / LZMA

Principe : L'algorithme LZMA (Lempel-Ziv-Markov chain Algorithm) utilise un dictionnaire de grande taille et un codage arithmétique.
Capacités :

— Excellent ratio de compression (meilleur que DEFLATE)

— Dictionnaires jusqu'à 4 GB

— Support multi-fichiers avec chiffrement AES-256

— Open source

Limites :

— Très lent en compression (10-20× plus lent que DEFLATE)

— Forte consommation mémoire

— Décompression également plus lente

— Moins universel que ZIP

2.4 Zstandard (Zstd)

Zstandard

Principe : Algorithme moderne développé par Facebook, combinant LZ77 et des techniques de prédiction avancées.
Capacités :

— Excellent compromis vitesse/ratio

— Très rapide en décompression (500+ MB/s)
— Niveaux de compression très larges (1-22)
— Support streaming et compression parallèle
— Dictionnaires d'entraînement pour fichiers similaires

Limites :

— Moins universel que ZIP (adoption récente)
— Pas de format d'archive standard (nécessite TAR)
— Complexité d'implémentation élevée

2.5 Brotli

Brotli

Principe : Développé par Google pour la compression web, basé sur LZ77, Huffman et un dictionnaire pré-défini.

Capacités :

— Meilleur ratio que GZIP sur texte/HTML/CSS
— Dictionnaire statique optimisé pour le web
— Standard HTTP/2

Limites :

— Très lent en compression niveau maximum
— Optimisé pour le web (moins efficace sur données binaires)
— Pas de format d'archive

3 Choix Technologique Retenu

Après analyse comparative, nous avons décidé d'implémenter notre compresseur en utilisant **l'algorithme DEFLATE** comme base, pour les raisons suivantes :

3.1 Justification du choix

1. **Équilibre optimal** : DEFLATE offre un excellent compromis entre ratio de compression, vitesse et complexité d'implémentation, adapté au cadre d'un projet pédagogique.
2. **Compréhension approfondie** : L'implémentation de DEFLATE nécessite la maîtrise de deux algorithmes fondamentaux (LZ77 et Huffman), objectif pédagogique central de ce projet.
3. **Performance acceptable** : Contrairement à LZMA, DEFLATE reste suffisamment rapide pour permettre des tests itératifs et une utilisation pratique.
4. **Bibliothèques Rust** : L'écosystème Rust propose `flate2`, une implémentation optimisée que nous pourrons utiliser ou étudier.

5. **Différenciation par le format** : Notre valeur ajoutée ne réside pas dans l'algorithme lui-même, mais dans la conception d'un format d'archive personnalisé `.mal` avec des fonctionnalités modernes (multi-fichiers, CRC32, métadonnées riches, visualisation temps réel).

3.2 Technologies complémentaires

— **Langage** : Rust — pour sa sécurité mémoire, ses performances et sa gestion d'erreurs robuste

— **Intégrité** : CRC32 — checksum rapide et efficace pour détecter les corruptions

— **Sérialisation** : JSON/Bincode — pour les métadonnées et l'index de fichiers

— **Interface** : CLI avec `clap` et `indicatif` — pour une expérience utilisateur moderne

4 Répartition des Tâches et Planning

Le projet sera développé en deux phases distinctes, alignées sur les soutenances. Le tableau suivant détaille la répartition équilibrée des responsabilités entre les membres de l'équipe.

4.1 Phase 1 : Jusqu'à la première soutenance

Cette phase se concentrera sur l'implémentation des fonctionnalités de base du compresseur.

Responsable	Tâches
Léos	— Mise en place de l'architecture du projet Rust — Implémentation de la structure du format `.mal` (header, métadonnées) — Développement du module de sérialisation/désérialisation — Écriture et lecture des fichiers d'archive
Adrien	— Intégration de l'algorithme DEFLATE (via `flate2`) — Implémentation du calcul CRC32 pour chaque fichier — Développement de la commande `compress` avec niveaux — Gestion des erreurs et validation des entrées
Maxime	— Développement de la CLI avec `clap` — Implémentation de la commande `decompress` — Collecte récursive des fichiers et dossiers — Tests unitaires et validation du pipeline complet

Table 1 – Répartition des tâches — Phase 1

Livrables Phase 1 :

— Compression d'un ou plusieurs fichiers vers `.mal`

— Décompression d'une archive `.mal`

— Vérification CRC32 basique
— Interface CLI fonctionnelle
— Documentation technique initiale

4.2 Phase 2 : Jusqu'à la deuxième soutenance

Cette phase finalisera le projet avec les fonctionnalités avancées et l'optimisation.

Responsable	**Tâches**
Léos	— Implémentation de la commande `list` avec affichage formaté — Optimisation du format d'archive (index, offsets) — Ajout de métadonnées enrichies (timestamps, permissions) — Benchmarks et mesures de performance
Adrien	— Implémentation de la commande `verify` complète — Amélioration de la gestion d'erreurs (messages détaillés) — Optimisation de la compression (streaming, buffers) — Documentation algorithmique (LaTeX)
Maxime	— Implémentation des barres de progression (`indicatif`) — Ajout de la colorisation des sorties (`colored`) — Tests d'intégration complets et edge cases — Rédaction du README et guide utilisateur

Table 2 – Répartition des tâches — Phase 2

Livrables Phase 2 :
— Toutes les commandes implémentées (`compress`, `decompress`, `list`, `verify`)
— Barres de progression en temps réel
— Vérification d'intégrité robuste
— Tests exhaustifs avec couverture $> 80\%$
— Documentation complète (technique et utilisateur)
— Présentation et démonstration finale

5 Conclusion

Ce projet nous permettra d'explorer en profondeur les algorithmes de compression sans perte, tout en développant une solution concrète et utilisable. En choisissant DEFLATE comme base algorithmique et en concevant un format d'archive personnalisé, nous équilibrons apprentissage théorique et application pratique.

La répartition équitable des tâches et le planning détaillé nous donnent un cadre structuré pour mener à bien ce projet ambitieux. Nous sommes confiants dans notre capacité à livrer un compresseur `.mal` fonctionnel, performant et bien documenté d'ici la deuxième soutenance.

Ce cahier des charges constitue notre feuille de route pour les semaines à venir. Nous nous engageons à respecter ces objectifs tout en restant flexibles face aux imprévus techniques inhérents à tout projet de développement.